

Orchard Guide

The Zcash Orchard shielded-payment protocol

m@rek.onl

Abstract

This is the fourth and final volume of a four-part series — *Math Guide*, *Crypto Guide*, *Halo 2 Guide*, *Orchard Guide*. Assuming the three companion volumes, it develops Zcash’s Orchard shielded-payment protocol: notes and note commitments, the commitment tree and the procedure by which a wallet obtains a membership path, nullifiers, the key hierarchy, value commitments and the RedPallas signature, note encryption and trial decryption, the formal Action statement the SNARK proves, and the end-to-end security reductions that convert knowledge soundness into balance, nullifier uniqueness, and zero knowledge — closing with the Zcash proof-system lineage.

Contents

1	Introduction	3
2	The Orchard application: the relation proved by the SNARK	3
2.1	The problem and the four requirements on a shielded payment	3
2.2	Preliminaries: hashing and encoding on the curve	4
2.3	The commitment primitive: Sinsemilla	5
2.4	Keys: capabilities for spending and for viewing	6
2.5	Representation of a note: notes and note commitments	10
2.6	Proof of ownership of <i>some</i> valid note: the commitment tree	10
2.7	Prevention of double-spending: nullifiers	12
2.8	Transmission and detection of a note: encryption, trial decryption, and synchroni- sation	13
2.9	Conservation of value: value commitments and the binding signature	14
2.10	Authorisation of the spend: RedPallas	14
2.11	A single shielded payment, end to end	15
2.12	What a node verifies, without ever observing the note	16
2.13	The Action statement, formally	16
2.14	The Action circuit: arithmetisation of the relation	17
3	End-to-end security: derivation of Orchard’s properties from the SNARK	18
3.1	Guarantees provided by a valid Action proof	18
3.2	Reduction of Sinsemilla collision-resistance to DL on Pallas	18
3.3	Nullifier uniqueness	19
3.4	RedPallas unforgeability and re-randomisation	19
3.5	Balance preservation via the Pedersen homomorphism	19
3.6	Zero knowledge of the SNARK	19
3.7	Composition of end-to-end soundness	20
4	The Zcash proof-system landscape	20
4.1	Sprout (2016–present, deprecated)	21
4.2	Sapling (2018–present)	21
4.3	Orchard (2021–present, NU5)	21
4.4	Comparative summary	22

1 Introduction

This is the *Orchard Guide*, the fourth and final volume of a development of the cryptography behind Zcash’s Orchard protocol. It assumes the three companion volumes: the *Math Guide* (the underlying mathematics), the *Crypto Guide* (hash functions, commitments, signatures, key agreement, zero knowledge, and the remainder of the primitive toolbox), and the *Halo 2 Guide* (the general-purpose proof system). Halo 2 has no notion of money; this volume converts it into a shielded-payment protocol by fixing one specific relation — the *Action statement* — that every transaction must prove.

We develop the protocol from the problem it solves rather than from the symbols it uses. After we fix the curve-level preliminaries and the Sinsemilla hash, we develop each of the following as the answer to one sub-problem: how a unit of value is represented (notes and note commitments); how an owner proves ownership of a real, unspent note without revealing which one (the commitment tree and a membership path); how the protocol prevents double-spending (nullifiers); how value is conserved without disclosure (value commitments and the binding signature); how a node authorises a spend (RedPallas); and how a sender privately delivers and a recipient detects a note (note encryption and trial decryption). We then state the Action relation formally, show the procedure by which a node verifies it without ever seeing the note, trace the end-to-end security reductions, and close the volume with the Zcash proof-system lineage from Sprout through Sapling to Orchard.

2 The Orchard application: the relation proved by the SNARK

Halo 2 is a general-purpose proving system; it has no notion of money. The Orchard application converts it into a shielded-payment protocol by fixing one specific relation — the *Action statement* — that every transaction must prove. This section develops that relation from the problem it solves rather than from the symbols it uses. It begins from the question of what a private payment must accomplish, derives each cryptographic object as the answer to one sub-problem, then traces a complete payment from Alice to Bob end to end, and only then states the Action relation formally.

2.1 The problem and the four requirements on a shielded payment

A transparent ledger (Bitcoin, Zcash’s transparent pool) records, for every unspent output, its owner and its denomination (the number of units it holds), and a payment publicly transfers an output from one owner to another. A *shielded* payment must achieve the same effects — the transfer of value, the prevention of double-spending, and the conservation of total supply — while disclosing none of them. A private payment system must accordingly satisfy four requirements simultaneously:

1. *Represent a unit of value* so that its owner, value, and very existence stay hidden, yet its owner can later prove possession of it. (Notes and *note commitments* solve this, §2.5.)
2. *Prove possession of a real, unspent note* without identifying which note. (The *commitment tree* together with a membership proof solves this, §2.6.)
3. *Prevent double-spending* without a public list of which notes an owner spent. (*Nullifiers* solve this, §2.7.)
4. *Prove that no one created value* without revealing any amount. (*Value commitments* and the *binding signature* solve this, §2.9.)

These four requirements constitute the core of the protocol, and they rest on shared machinery. To render the section self-contained, we fix the build order by dependency — nothing appears before we define it — and lay it out here as a map:

1. **Preliminaries** (§2.2): the curve-level primitives every later object invokes — the hash-to-curve **GroupHash**, the coordinate map **Extract**, the point encoding $\star$, and domain separators.
2. **Sinsemilla** (§2.3): the commitment/hash built on those primitives, which both the note commitment and the Merkle tree use.
3. **Keys** (§2.4): the key hierarchy, from which the protocol derives addresses, commitments, and nullifiers.
4. **The four requirements in turn**: notes (§2.5), the commitment tree (§2.6), nullifiers (§2.7), and value commitments (§2.9); together with *RedPallas* (§2.10), the signature that authorises a spend.
5. **Assembly** (§2.11): a complete Alice-to-Bob payment, end to end.
6. **The Action statement** (§2.13): the formal relation, the conjunction of in-circuit checks that makes all four hold at once.

Each later object depends only on earlier ones; the dependency root is the preliminaries below.

2.2 Preliminaries: hashing and encoding on the curve

The protocol builds every Orchard object below from a small set of curve-level primitives. We define them once, here, before anything uses them: a hash-to-curve **GroupHash**, the coordinate map **Extract**, the point encoding $\star$, and the domain separators that keep distinct uses independent. All operate on the Pallas group $\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$.

The coordinate map **Extract.** $\text{Extract} : \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}}) \rightarrow \mathbb{F}_{p_{\text{Pallas}}}$ maps a curve point to its x -coordinate, $\text{Extract}(x, y) := x$ (with $\text{Extract}(\mathcal{O}) := 0$ by convention). It appears wherever a later object must be a *field element* rather than a curve point — because the Merkle tree, the nullifier set, and the proof’s public inputs are all field elements. It is not injective ($\pm(x, y)$ share an x), which is harmless here: the protocol only ever requires the x -coordinate as a binding digest, never to recover the point.

Point encoding: the $\star$ superscript. $P^\star \in \{0, 1\}^{256}$ denotes the canonical bit-encoding of a point $P \in \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$: its x -coordinate (an element of $\mathbb{F}_{p_{\text{Pallas}}}$, which fits in 255 bits, since $p_{\text{Pallas}} < 2^{255}$) packed into a 256-bit string together with one sign bit, placed in the otherwise-unused top bit, recording which of the two square roots $\pm y$ is meant. Unlike **Extract**, this encoding is injective — it determines P completely — and it is what we feed, as a bit-string, into a hash or commitment.

The hash-to-curve **GroupHash.** $\text{GroupHash}^{\mathbb{G}} : \{0, 1\}^* \times \{0, 1\}^* \rightarrow \mathbb{G}$ takes a pair (domain separator D , message M) and returns a point of $\mathbb{G}$. It is *deterministic* (the same input always yields the same point), *public* (anyone can evaluate it), and *efficiently computable*. It is the composition of two stages, $\text{GroupHash}^{\mathbb{G}}(D, M) := \text{map}_{\mathbb{G}}(\text{hash}(D, M))$, where hash maps the input to a base-field element (via BLAKE2s with D as personalisation) and $\text{map}_{\mathbb{G}} : \mathbb{F}_p \rightarrow \mathbb{G}$ deterministically encodes that field element as a curve point. The IETF hash-to-curve standard fixes the explicit form of $\text{map}_{\mathbb{G}}$ — the simplified SWU map composed with an isogeny; for everything that follows, we use only the properties that **GroupHash** is deterministic, public, and behaves like a random choice of point.

That last property is the purpose of the construction: because both stages are public and deterministic, the public strings D, M pin down the output $\text{GroupHash}^{\mathbb{G}}(D, M)$, so no party can have chosen it to satisfy a hidden relation such as $P_2 = [s]P_1$ for a secret s . We call such points *nothing-up-my-sleeve* generators. For the security arguments we *model* **GroupHash** as a random

oracle into $\mathbb{G}$ (the *Crypto Guide*): we treat its outputs as independent uniform points, so the family it produces has no efficiently-findable non-trivial discrete-log relation $\sum_i [a_i] P_i = \mathcal{O}$ — precisely the assumption the Sinsemilla binding and collision-resistance reductions invoke (§3.2). As with all random-oracle arguments this is a strong, standard, but unprovable modelling heuristic about the concrete hash, not a theorem.

Domain separators. A *domain separator* $D \in \{0,1\}^*$ is a fixed ASCII byte string naming the use site, fed to `GroupHash` so that logically distinct uses draw independent, unrelated generators (a collision found in one context cannot be transported to another). The strings are part of the protocol specification; the ones relevant here are

<code>z.cash : Orchard-NoteCommit</code>	note commitments (§2.5),
<code>z.cash : Orchard-CommitIvk</code>	the ivk commitment (§2.4),
<code>z.cash : Orchard-MerkleCRH</code>	the Merkle-tree node hash (§2.6),
<code>z.cash : Orchard-cv</code>	the value-commitment bases (§2.9),
<code>z.cash : SinsemillaQ, z.cash : SinsemillaS</code>	the Sinsemilla Q point and S table (§2.3).

2.3 The commitment primitive: Sinsemilla

Two of the objects we build — the note commitment (§2.5) and the Merkle-tree node hash (§2.6) — require the same primitive: a commitment/hash that the spender must recompute *inside the zero-knowledge circuit*. In-circuit cost is therefore the binding constraint, and it is what Sinsemilla (Bowe and Hopwood) is designed to minimise. Its collision resistance reduces to DL on a prime-order curve, so it inherits the same hardness assumption as everything else in Orchard rather than introducing a new one. It builds directly on the preliminaries of §2.2: its generators are `GroupHash` outputs, and its short form returns an `Extracted` coordinate.

Construction 2.1 (Sinsemilla). Fix a prime-order group $\mathbb{G}$ (Orchard: $\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$) and a chunk width k ($k = 10$ in Orchard). From the domain separator D , derive the $2^k + 1$ Sinsemilla generators:

$$\begin{aligned} Q(D) &:= \text{GroupHash}^{\mathbb{G}}(\text{z.cash} : \text{SinsemillaQ}, D) \in \mathbb{G}, \\ P[m] &:= \text{GroupHash}^{\mathbb{G}}(\text{z.cash} : \text{SinsemillaS}, \text{LE}_{32}(m)) \in \mathbb{G}, \quad 0 \leq m < 2^k. \end{aligned}$$

The point $Q(D)$ is the domain-dependent starting accumulator; the 2^k points $P[0], \dots, P[2^k - 1]$ form a fixed table indexed by a k -bit chunk value, shared across all Sinsemilla domains. For a message M split into k -bit chunks $m_1, \dots, m_\ell$ (so each $m_i \in \{0, \dots, 2^k - 1\}$ indexes the table),

$$\text{Sinsemilla}_D(M) := \text{Acc}_\ell, \quad \text{Acc}_0 := Q(D), \quad \text{Acc}_{i+1} := (\text{Acc}_i \dot{+} P[m_{i+1}]) \dot{+} \text{Acc}_i,$$

with $\dot{+}$ *incomplete* short-Weierstrass addition. The output $\text{Acc}_\ell \in \mathbb{G}$ is a curve point.

Theorem 2.2 (Sinsemilla collision resistance). *For fixed-length inputs, Sinsemilla_D is collision-resistant assuming DL on $\mathbb{G}$ and modelling the generator derivation as a random oracle. (We prove this in §3.2.)*

Definition 2.3 (Sinsemilla commitment and short commitment). The *Sinsemilla commitment* adds a hiding term to the hash:

$$\text{SinsemillaCommit}_r(D, M) := \text{Sinsemilla}_D(M) \dot{+} [r] H_D \in \mathbb{G},$$

where $r \in \mathbb{F}_{p_{\text{Vesta}}}$ is the commitment randomness (trapdoor) and $H_D \in \mathbb{G}$ is a fixed blinding generator for the domain D , derived as the `GroupHash` output

$$H_D := \text{GroupHash}^{\mathbb{G}}(D, \mathbf{r}) \in \mathbb{G}$$

(a nothing-up-my-sleeve point, §2.2). Because H_D is a `GroupHash` output, no party knows a discrete-log relation between it and the generators $Q(D), P[m]$ that `SinsemillaD` uses; this independence is exactly what makes the $[r]H_D$ term a hiding blinding factor whose presence no one can forge, the same role the independent base H plays in a Pedersen commitment (the *Crypto Guide*). The *short commitment* returns only its x -coordinate, a base-field element rather than a curve point:

$$\text{ShortCommit}_r(D, M) := \text{Extract}(\text{SinsemillaCommit}_r(D, M)) \in \mathbb{F}_{p_{\text{Pallas}}},$$

with `Extract` the coordinate map of §2.2.

The commitment is perfectly hiding and computationally binding — the two properties §2.5 requires of `NoteCommit`. Indeed the note commitment `NoteCommitrcm(·)` is exactly `SinsemillaCommitrcm` under the domain `z.cash:Orchard-NoteCommit`. The short form `ShortCommit` appears wherever the consumer requires a field element instead of a point — the Merkle-tree node hash (§2.6) and the incoming viewing key `ivk` (§2.4). When we write `ShortCommit` with two message arguments, as in `ShortCommitivk(ak*, nk)`, the message M is their bit-string concatenation $ak^* \parallel nk$; for `ivk` the domain is `z.cash:Orchard-CommitIvk` and the result lies in $\{1, \dots, p_{\text{Pallas}} - 1\}$ (we exclude the zero output, which cannot occur for a valid key, so that `ivk` is a usable scalar).

In-circuit cost. Each round consumes a whole k -bit chunk via one table lookup of $P[m]$ and two incomplete additions, so a 510-bit message costs 51 rounds rather than 510 bit-operations. In a PLONKish circuit with lookups (the *Halo 2 Guide*) this is dramatically cheaper than Sapling’s bit-by-bit Pedersen hash — which is the reason the protocol changed hash between generations.

2.4 Keys: capabilities for spending and for viewing

We build the Orchard machinery bottom-up, beginning with the keys, because everything later — addresses, note commitments, nullifiers — derives from them. A practical requirement shapes the key hierarchy: different parties should be able to perform different operations. The owner can spend. An auditor should be able to *see* incoming and outgoing payments without being able to spend. A point-of-sale terminal should be able to *detect* incoming payments without being able to see outgoing ones. Each of these is a distinct capability, and the key tree makes each capability a separate derived key — the owner can hand out a lower-level key without surrendering the powers above it.

Definition 2.4 (Orchard key hierarchy). The root secret is a 32-byte *spending key* sk , chosen uniformly at random when the owner creates the account (in a deployed wallet it typically derives from a seed phrase via ZIP-32 hierarchical-deterministic derivation, but for present purposes it is simply a uniform 256-bit string, $sk \in \{0, 1\}^{256}$). Every other key is a deterministic function of sk .

Expansion function. The expansion uses BLAKE2b, a conventional (non-arithmetisation-friendly) cryptographic hash; “BLAKE2b-512” denotes the instance with a 512-bit (64-byte) digest. BLAKE2b natively supports a *keyed* mode: it takes an optional secret key $K \in \{0, 1\}^{\leq 512}$ alongside the message m and computes a keyed digest $\text{BLAKE2b-512}(K, m)$ (internally, it pads the key to a full block and prepends it to the message), so that without knowledge of K the output is pseudorandom. This is why we write the function with two arguments: the first slot is the key, the second the message. Using it in keyed mode is what converts a plain hash into a *pseudorandom function* (PRF) — a keyed family $\{f_K\}_K$ whose outputs are, for a secret uniform K , computationally indistinguishable from a truly random function of the message.

Accordingly, write $\text{PRFexpand} : \{0, 1\}^{256} \times \{0, 1\}^* \rightarrow \{0, 1\}^{512}$ for the PRF instantiated as $\text{PRFexpand}(sk, t) := \text{BLAKE2b-512}(\text{``Zcash_ExpandSeed''}, sk \parallel t)$ — unkeyed BLAKE2b-512 with the fixed 16-byte personalisation string ```Zcash_ExpandSeed''` and input $sk \parallel t$, where the secret sk acts as the PRF key and the leading bytes of t are a domain-separation tag identifying which sub-secret it produces. (We write the key as a subscript, $\text{PRFexpand}_{sk}(t)$, when convenient.)

We reduce its 512-bit output into the required field or scalar set. From sk derive the three spend-side secrets

$$\begin{aligned}\text{ask} &:= \text{ToScalar}(\text{PRFexpand}_{\text{sk}}(0\text{x}06)) \in \mathbb{F}_{p_{\text{Vesta}}}, \\ \text{nk} &:= \text{ToBase}(\text{PRFexpand}_{\text{sk}}(0\text{x}07)) \in \mathbb{F}_{p_{\text{Pallas}}}, \\ \text{r}_{\text{ivk}} &:= \text{ToScalar}(\text{PRFexpand}_{\text{sk}}(0\text{x}08)) \in \mathbb{F}_{p_{\text{Vesta}}},\end{aligned}$$

so that the *spend authorising key* ask and the *CommitIvk randomness* r_{ivk} are scalars in $\mathbb{F}_{p_{\text{Vesta}}}$ (the scalar field of Pallas, i.e. the integers mod the Pallas group order p_{Vesta}), while the *nullifier key* nk is a base-field element in $\mathbb{F}_{p_{\text{Pallas}}}$. The two reduction maps read the 512-bit PRF output as an integer and reduce it modulo the relevant prime:

$$\begin{aligned}\text{ToScalar}(x) &:= \text{LEOS2IP}_{512}(x) \bmod p_{\text{Vesta}} \in \mathbb{F}_{p_{\text{Vesta}}}, \\ \text{ToBase}(x) &:= \text{LEOS2IP}_{512}(x) \bmod p_{\text{Pallas}} \in \mathbb{F}_{p_{\text{Pallas}}},\end{aligned}$$

where LEOS2IP_{512} is the little-endian octet-string-to-integer map sending a 64-byte string to the integer $\sum_{i=0}^{63} x_i 256^i \in \{0, \dots, 2^{512} - 1\}$. ToScalar lands in $\mathbb{F}_{p_{\text{Vesta}}}$, ToBase in $\mathbb{F}_{p_{\text{Pallas}}}$; the two targets differ only because the protocol uses nk as a field element (it feeds it to the nullifier PRF, §2.7) whereas it uses $\text{ask}, \text{r}_{\text{ivk}}$ as scalars multiplying group elements. Because the input is 512 bits wide while each modulus is just over 2^{254} (about 2^{254} , of bit length 255, hence under 2^{255}), the reduction is statistically indistinguishable from uniform on the target set (the modular bias is $O(2^{254}/2^{512}) = O(2^{-258})$, negligible). The distinct tag bytes $0\text{x}06, 0\text{x}07, 0\text{x}08$ make the three outputs independent.

Spend validating key. Let $G^{\text{SpendAuth}} \in \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$ be a fixed, publicly-known generator of the Pallas group, obtained by $G^{\text{SpendAuth}} := \text{GroupHash}^{\text{Pallas}}(\text{z.cash} : \text{Orchard}, G)$ (a nothing-up-my-sleeve point, §2.3). The *spend validating key* is then the public point

$$\text{ak} := [\text{ask}] G^{\text{SpendAuth}} \in \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}}).$$

The *full viewing key* bundles the three quantities needed to recognise and decrypt one's own notes, $\text{fvk} := (\text{ak}, \text{nk}, \text{r}_{\text{ivk}}) \in \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}}) \times \mathbb{F}_{p_{\text{Pallas}}} \times \mathbb{F}_{p_{\text{Vesta}}}$. From it derive

$$\begin{aligned}\text{ivk} &:= \text{ShortCommit}_{\text{r}_{\text{ivk}}}(\text{ak}^*, \text{nk}) \in \{1, \dots, p_{\text{Pallas}} - 1\} \subset \mathbb{F}_{p_{\text{Pallas}}}, \\ K &:= \text{I2LEBSP}_{256}(\text{r}_{\text{ivk}}), \\ R &:= \text{PRFexpand}(K, 0\text{x}82 \parallel \text{I2LEOSP}_{256}(\text{ak}) \parallel \text{I2LEOSP}_{256}(\text{nk})) \in \{0, 1\}^{512}, \\ (\text{dk}, \text{ovk}) &:= (R_{[0..256)}, R_{[256..512)}) \in \{0, 1\}^{256} \times \{0, 1\}^{256},\end{aligned}$$

where ShortCommit is from Definition 2.3. This yields the *incoming* viewing key ivk (a base-field scalar), the *diversifier key* dk , and the *outgoing* viewing key ovk . The dk/ovk derivation is precisely the following: we split a *single* 512-bit PRFexpand output R , keyed by r_{ivk} over the input $0\text{x}82 \parallel \text{ak} \parallel \text{nk}$ (not over an empty message), into its two 256-bit halves — dk the first, ovk the second. The diversifier d below is *not* part of this split; it derives afterwards from dk . Here ak^* is the $\star$ -encoding of ak (§2.2), while I2LEOSP_{256} denotes the 32-byte little-endian encoding of a field element or point. A *diversified address* is, for any index $d_{\text{plain}} \in \{0, 1\}^{88}$,

$$\begin{aligned}d &:= \text{FF1-AES256}_{\text{dk}}(d_{\text{plain}}) \in \{0, 1\}^{88}, \\ \text{g}_d &:= \text{GroupHash}^{\text{Pallas}}(\text{z.cash} : \text{Orchard-gd}, d) \in \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}}), \\ \text{pk}_d &:= [\text{ivk}] \text{g}_d \in \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}}),\end{aligned}$$

where $\text{FF1-AES256}_{\text{dk}}$ produces the *diversifier* $d \in \{0, 1\}^{88}$, the NIST FF1 format-preserving encryption mode (built on AES-256) keyed by dk : a keyed *permutation* of the 88-bit index space $\{0, 1\}^{88}$, so that each d_{plain} maps to a distinct 88-bit diversifier d and different d_{plain} give unlinkable-looking addresses, all derived from the one key. The *address* is the pair $\text{addr} := (d, \text{pk}_d) \in \{0, 1\}^{88} \times \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$.

Choosing the index d_{plain} . The receiving wallet chooses the index freely: *any* of the 2^{88} values yields a valid, spendable address for the same ivk . No derivation fixes it and the wallet must avoid no value — it allocates indices on demand, conventionally sequentially from $d_{\text{plain}} = 0$ (so index 0 is the *default diversifier*), assigning a fresh index per payee or per invoice to keep their addresses mutually unlinkable; selecting indices uniformly at random is equally valid and avoids leaking a count. Every index works for a reason specific to Orchard: the hash-to-curve $\text{GroupHash}^{\text{Pallas}}$ of §2.2 is the simplified-SWU map, which returns a valid point for every input, so $\mathbf{g}_d$ is always a well-defined group element (and never $\mathcal{O}$, as the Pallas group has prime order). This is a deliberate improvement over Sapling, where the diversifier hash failed on roughly half of all indices and the wallet had to increment d_{plain} until it found a valid one; in Orchard no such rejection loop exists.

Rationale for routing the index through AES. The step from d_{plain} to d is a keyed *permutation* (format-preserving encryption), not a relabelling, for three reasons that no simpler map satisfies together. (i) *Unlinkability*: the wallet hands out indices in order $0, 1, 2, \dots$, so if $d = d_{\text{plain}}$ the diversifiers in a wallet’s addresses would be consecutive integers — revealing that they share an owner and how many it has issued; encrypting under dk scatters them pseudorandomly over $\{0, 1\}^{88}$, so each d looks uniform and uncorrelated. (ii) *Bijection*: a permutation never collides two indices onto one d , and is *invertible* — with dk the wallet decrypts d back to its index, so it can re-derive any address from the seed and recover which index an address used, storing no per-address secret. A plain hash would be neither collision-free nor invertible. (iii) *Format-preserving*: the address format fixes the diversifier at exactly 88 bits, and FF1 maps $\{0, 1\}^{88} \rightarrow \{0, 1\}^{88}$ bijectively, which a truncated hash cannot. AES-256 is merely the standard block cipher underlying NIST FF1; this derivation runs *wallet-side, never in the circuit*, so — unlike the note commitment and Merkle hash, which must be SNARK-cheap (Sinsemilla, Poseidon) — a conventional fast cipher is appropriate here, requiring no arithmetisation-friendliness.

The structure is a deliberate capability ladder (Figure 1), each rung conferring strictly less power than the one above:

- sk / ask — *spend*. Whoever holds ask can authorise spends; it sits at the top and nothing below it can recover it.
- fvk (hence ivk, ovk) — *see everything*. Detects incoming notes (via ivk , which recovers pk_d) and views outgoing notes (via ovk), but cannot spend, because deriving fvk from sk is one-way.
- ivk alone — *detect incoming*. Sufficient to recognise notes addressed to the holder and scan the chain, nothing more — the key suitable for an always-online wallet server.
- A diversified address (d, pk_d) — *receive*. A public handle anyone can pay; one ivk generates unboundedly many unlinkable addresses, all spendable by the same key.

Of these, only the diversified address is *public*. The viewing keys $\text{fvk}, \text{ivk}, \text{ovk}$ are *secret* key material — “viewing” names what they let the holder *do* (see transactions), not that one may publish them. Disclosing a viewing key confers on the recipient the ability to deanonymise every payment to or from the wallet, so the owner shares it only deliberately (with an auditor, say) and otherwise guards it like any other secret. The ladder is useful precisely because these capabilities are *separable*: one can delegate “see incoming” (ivk) without delegating “see everything” (fvk), and neither confers the ability to spend.

Two questions of motivation that the bare formulas do not answer. *Why is ivk a commitment, $\text{ShortCommit}_{\text{ivk}}(\text{ak}^*, \text{nk})$, rather than a plain hash?* Because ivk must bind *both* ak (spend authority) and nk (nullifier derivation) to one identity: a commitment lets the in-circuit spender prove $\text{pk}_d = [\text{ivk}]\mathbf{g}_d$ and that the same ivk ties to the ak and nk used elsewhere in the same proof (checks A5–A7 below), which a one-way hash would not permit it to open algebraically. *Why is nk separate from ask ?* So that the owner can delegate the capability to compute nullifiers (which

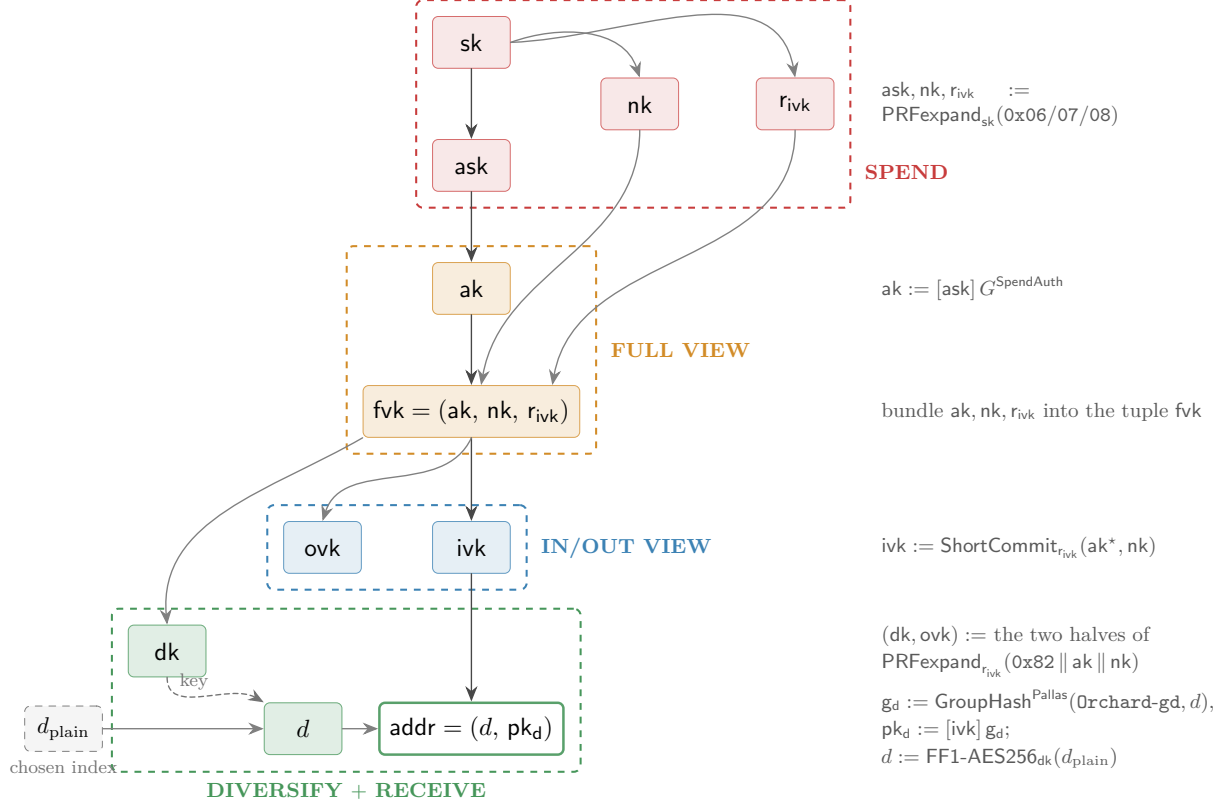


Figure 1: The Orchard key hierarchy as a capability ladder, read top to bottom. Each arrow is a one-way derivation; the right-hand gutter shows the operation that produces the child from its parent(s). The dk and ovk keys are the two halves of one $\text{PRFexpand}_{r_{ivk}}$ output (tag $0x82$, input $ak \parallel nk$); they play different roles and land in different tiers — ovk is the *outgoing* viewing key, grouped with the *incoming* viewing key ivk as the IN/OUT viewing tier, whereas dk is the *diversifier* key, which sits in the address layer because it generates diversified addresses. Concretely d is the format-preserving encryption FF1-AES256 of a chosen index d_{plain} under the key dk (the dashed edge marks dk entering as the cipher key), and $pk_d = [ivk] g_d$, so the address binds both keys. *Every node is secret key material (shaded) except the diversified address (unshaded), the only value the protocol ever publishes* — in particular the viewing keys fvk , ivk , ovk are *secret*: “viewing” names what they let the holder do, not that the protocol publishes them. Each dashed tier holds a strictly weaker capability than the one above, so a holder may receive a lower-tier key without gaining the powers above it. Drawn to match Definition 2.4.

a viewing key needs to determine which of the holder’s notes are already spent) *without* conferring the ability to spend — the viewing key holds nk but never ask .

2.5 Representation of a note: notes and note commitments

The first requirement is to represent a note such that it stays invisible on-chain yet provable by its owner. The protocol therefore never places the note on the chain at all; it publishes only a hiding commitment to it.

Definition 2.5 (Orchard note). An Orchard *note* — the private object representing one unit of value — is a tuple

$$\mathbf{n} := (\text{addr}, v, \rho, \psi, \text{rcm}),$$

where $\text{addr} := (d, \text{pk}_d)$ is the recipient’s diversified address; $v \in \{0, \dots, 2^{64} - 1\}$ is the value in zatoshi; $\rho \in \mathbb{F}_{p_{\text{Pallas}}}$ is a per-note uniqueness anchor; $\psi \in \mathbb{F}_{p_{\text{Pallas}}}$ is sender-chosen randomness; and $\text{rcm} \in \mathbb{F}_{p_{\text{Vesta}}}$ is the commitment trapdoor.

The protocol never publishes the note itself. What it records on-chain is its *note commitment*

$$\text{cm} := \text{NoteCommit}_{\text{rcm}}(\mathbf{g}_d^\star \parallel \text{pk}_d^\star \parallel \text{LE}_{64}(v) \parallel \text{LE}_{255}(\rho) \parallel \text{LE}_{255}(\psi)) \in \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}}),$$

a commitment that is *perfectly hiding* (the chain learns nothing about $\text{addr}, v, \rho, \psi$ from cm) and *computationally binding* (the sender cannot later claim the commitment was to a different note). These are precisely the two properties of a commitment scheme from the *Crypto Guide*; here the scheme is Sinsemilla (§2.3), chosen because it is inexpensive to recompute *inside the circuit*, which the spender must do. The chain stores $\text{cmx} := \text{Extract}(\text{cm}) \in \mathbb{F}_{p_{\text{Pallas}}}$, the x -coordinate of the commitment point, because the Merkle tree consumes a field element.

The three address-related quantities appearing in NoteCommit are the key material that §2.4 defines: the diversifier $d \in \{0, 1\}^{88}$, the diversified base $\mathbf{g}_d \in \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$ (the per-address generator $\text{GroupHash}^{\text{Pallas}}(\text{z.cash} : \text{Orchard-gd}, d)$), and the transmission key $\text{pk}_d = [\text{ivk}] \mathbf{g}_d \in \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$ that identifies the recipient; the address is $\text{addr} = (d, \text{pk}_d)$. Recall the $\star$ -encoding from §2.2: $\mathbf{g}_d^\star, \text{pk}_d^\star \in \{0, 1\}^{255}$. The NoteCommit input is the bit-string concatenation ($\parallel$) of these point encodings with the little-endian integer encodings $\text{LE}_{64}(v) \in \{0, 1\}^{64}$ and $\text{LE}_{255}(\rho), \text{LE}_{255}(\psi) \in \{0, 1\}^{255}$.

The choice of fields is as follows. addr and v are the note’s content. ρ and ψ exist solely to render the eventual nullifier unique and unpredictable; their role becomes apparent only in §2.7: a note carries randomness whose purpose is downstream.

2.6 Proof of ownership of *some* valid note: the commitment tree

The second requirement is to prove the spent note is valid without revealing which note. When Alice spends, she must convince the network that the note she is spending is a valid note that someone actually created; otherwise she could spend a note she invented. On a transparent chain this is trivial: the output is present in the UTXO set. But Alice must prove it *without revealing which note*, because revealing the note would deanonymise the payment.

This is the canonical use case for a Merkle tree as a privacy set. The network maintains one append-only tree whose leaves are *every note commitment ever created*, in order. To prove that her note is valid, Alice proves *membership*: she exhibits a Merkle authentication path from her leaf cmx up to a root the network already trusts. The proof reveals that her note is *one of* the leaves, but a Merkle path inside the zero-knowledge circuit hides *which* leaf — the verifier checks the path arithmetic without seeing the position or the sibling values. The anonymity set is the entire tree: the larger it grows, the better the privacy.

Definition 2.6 (Orchard commitment tree). The Orchard *note commitment tree* is an append-only Merkle tree of fixed depth 32 (so up to 2^{32} notes). Leaves are cmx values in creation order. The parent of a left child ℓ and right child r at layer ℓ' is

$$\text{MerkleCRH}_{\ell'}^{\text{Orchard}}(\ell, r) := \text{ShortHash}_{\text{Orchard-MerkleCRH}}(\text{LE}_{10}(\ell') \parallel \ell \parallel r) \in \mathbb{F}_{p_{\text{Pallas}}},$$

a layer-tagged Sinsemilla short hash (so leaves cmx , internal nodes, and the root all live in $\mathbb{F}_{p_{\text{Pallas}}}$). We call a historic tree root an *anchor* (a single field element, $\text{anchor} \in \mathbb{F}_{p_{\text{Pallas}}}$); the network remembers recent anchors, and a spend may prove membership against any of them.

Two design details are significant. The tag carries the layer index ℓ' so that a path at one height cannot replay at another. And empty leaf positions take an “uncommitted” sentinel value 2: the choice is not arbitrary — $2^3 + 5 = 13$ is a non-square modulo both Pasta primes (the *Math Guide* verifies this), so 2 is not the x -coordinate of any curve point, hence can never equal a valid cmx and can never be silently confused with a genuine commitment.

Derivation of the authentication path. The path is *not* something Alice stored or had to track; it is a deterministic function of *public* data. The tree’s leaves are every cmx ever published, in order, and the path is the sibling at each level on the route from her leaf up to the root (Figure 2); and every one of those siblings is itself a hash of public leaves. Hence anyone holding the public commitments can *recompute* any note’s path, against any past root, at any time. The only secret is *which* leaf is hers, and that remains hidden because she proves the path in zero knowledge (check A3), not because the leaves are hidden.

Seen this way, a years-old note is no special case. To spend it Alice selects an *anchor* — a recent root the network already trusts — reads the public commitments that were in the tree at that anchor, and hashes them up to obtain her siblings. A freshly restored, long-offline wallet performs exactly this: it rebuilds the path from the chain. (The “incremental witness” a continuously-online wallet maintains as the tree grows is only an optimisation; a cold wallet recomputes from public data instead.) The wallet performs this *locally* and never requests a particular note’s path from a server, which would reveal which note is hers.

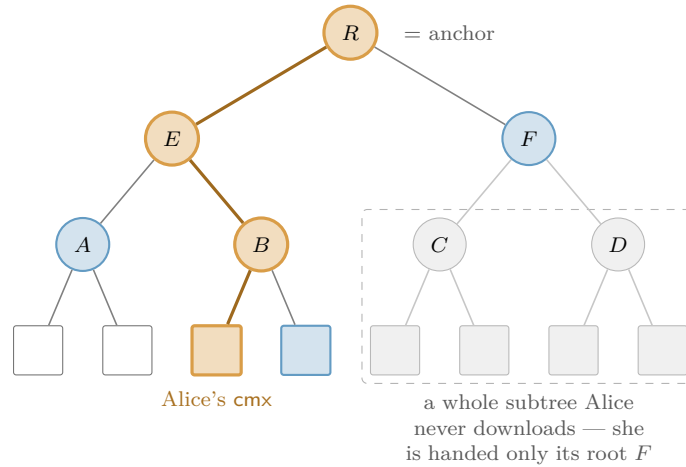


Figure 2: Computation of a Merkle authentication path (drawn at depth 3; Orchard’s tree has depth 32). The leaves are the public note commitments cmx , in order. To prove her leaf (orange) is in the tree, Alice recomputes the bold route up to the root R — the *anchor* — folding in, at each level, the sibling hash beside it: the *authentication path* A, L_3, F (blue). Every sibling is itself a hash of public leaves, so Alice recomputes the entire path from public data and stores nothing. Efficiency is the only reason not to hash the whole tree: a far subtree enters her path solely through its single root (here F), so she never downloads its leaves (faded) — the `GetSubtreeRoots` shortcut that lets even a freshly restored wallet witness a years-old note.

The one substantive difficulty is efficiency: the live tree holds up to 2^{32} leaves, and Alice wishes neither to download all of them nor to hash the whole tree for one path. This is unnecessary. Her note resides in a single small subtree (a “shard” of 2^{16} leaves in Orchard), and she needs only *that* shard’s leaves, to build the siblings nearest her own. Every *other* subtree — millions of leaves of no relevance to her — enters her path through its single *root* alone (the faded subtree of Figure 2,

folded into F). A light wallet therefore fetches only the precomputed roots of the other shards (GetSubtreeRoots), the leaves of its own shard, and a frontier checkpoint, and hashes them into the path. “Obtaining the path” is, end to end, *recomputation of it from public commitment data* — the protocol merely serves that data in a form that skips the irrelevant leaves. The result is the private witness for check A3; only the roots (anchors) are ever public.

Verification of an anchor as the tree grows. An *anchor* is the value of the Merkle root at some past moment — a single field element. The apparent difficulty is that the tree is append-only and continues to grow, so by the time Alice’s transaction is mined the live root has advanced. Three facts make proving against a fixed anchor both possible and sound:

- *A root is a fixed commitment to a fixed leaf set.* The root is a deterministic hash of all leaves present when someone took it. It does not change retroactively: appending new leaves later produces *new* roots but leaves every *past* root exactly as it was. Thus “the anchor” unambiguously denotes one historical state of the tree.
- *A membership path is relative to one root.* Alice’s authentication path proves cm_{old} is a leaf of the tree *whose root is that anchor*. Because her note arose before anyone took the anchor, such a path exists and she can compute it. The circuit (check A3) recomputes the root from her leaf and path and asserts it equals the public anchor — collision resistance of the layer-tagged hash (§2.3) renders forging a path to a root for a non-member infeasible.
- *Consensus accepts any historical main-chain root, not just recent ones.* A node treats a value as a valid anchor once it has appeared as the Orchard tree root at the end of some block on the node’s accepted main chain. There is *no* short sliding window: an anchor may be arbitrarily old, the only floor being the Orchard activation height (NU5, mainnet block 1,687,104) — one cannot anchor to a root that predates the tree’s existence. A spend names which anchor it used; the node accepts the proof iff that root belongs to a block it accepts. This fully decouples proving from broadcasting: Alice need not race the chain tip. The only *recency* caveat runs in the opposite direction — if she anchors to a very recent block that a later reorganisation removes, her anchor is no longer on the main chain and she must rebuild the transaction; anchoring at least 100 blocks deep (Zcash’s reorg/finality limit) avoids this entirely.

Crucially, proving against an old root is *not* a soundness hole: membership in an older, smaller tree implies the note was committed even earlier, which is precisely what Alice must establish. The nullifier mechanism (§2.7), not the anchor, is what prevents spending the same old note twice.

2.7 Prevention of double-spending: nullifiers

The third requirement: to prevent Alice from spending the same note twice, without a public registry of spent notes (which would leak exactly what is being hidden). Each note yields a single deterministic tag — its *nullifier* — that appears only when its owner spends the note. The network maintains a set of all revealed nullifiers and rejects any repeat. The nullifier must satisfy two properties that pull in opposite directions: it must be *deterministic* (so the same note always yields the same tag, making a second spend detectable) yet *unlinkable* (so that publishing it reveals nothing about which note or address it came from).

Definition 2.7 (Orchard nullifier). For a note $\mathbf{n}$ with commitment cm and the owner’s nullifier key nk ,

$$\text{nf} := \text{Extract}([F_{\text{nk}}(\rho) + \psi]G^{\text{nf}} + \text{cm}) \in \mathbb{F}_{p_{\text{Pallas}}},$$

where $F_{\text{nk}} : \mathbb{F}_{p_{\text{Pallas}}} \rightarrow \mathbb{F}_{p_{\text{Pallas}}}$ is a circuit-efficient PRF (Poseidon over the Pallas base field, the *Crypto Guide*), $G^{\text{nf}} := \text{GroupHash}^{\text{Pallas}}(\mathbf{z.cash} : \text{Orchard}, \mathbf{K}) \in \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$ is a fixed nothing-up-my-sleeve Pallas generator (§2.2), the scalar $F_{\text{nk}}(\rho) + \psi$ lives in $\mathbb{F}_{p_{\text{Pallas}}}$ (a valid Pallas scalar because

$p_{\text{Pallas}} < p_{\text{Vesta}}$), and **Extract** takes the x -coordinate. (We write G^{nf} for this base to distinguish it from the spend-auth generator $G^{\text{SpendAuth}}$ of §2.4.)

We read the construction against the two requirements. **Determinism**: every ingredient $(\text{nk}, \rho, \psi, \text{cm})$ is fixed once the note exists, so nf is a fixed function of the note — spending it twice yields the same nf twice, and the network rejects the second. **Unlinkability**: the term $[F_{\text{nk}}(\rho)]G$ masks the commitment with a value that appears random to anyone without nk , so no one can tie the published nf back to cm or to the address (this is where the note’s ρ and ψ perform their function). The security properties:

- *No double-spend* (the Orchard Book calls this *balance*, since double-spending is how one would create value): each note has exactly one nullifier, and forging a colliding nullifier for a different note reduces to DL on Pallas.
- *Spend unlinkability*: without nk , the published nf is unlinkable to the note or address, under the Decisional Diffie–Hellman (DDH) assumption on Pallas — that $([a]G, [b]G, [ab]G)$ is computationally indistinguishable from $([a]G, [b]G, [c]G)$ for independent uniform a, b, c — together with the PRF assumption on F .
- *Forward consistency (Faerie resistance)*: a freshly created note takes as its anchor field the nullifier of the note spent alongside it, $\rho_{\text{new}} := \text{nf}_{\text{old}}$; since the network already guarantees nullifiers are unique, this forces every note’s ρ to be globally unique, closing an attack from earlier designs in which forged notes could be made to share a nullifier.

2.8 Transmission and detection of a note: encryption, trial decryption, and synchronisation

The on-chain cmx conceals the note entirely, which leaves a question the commitment alone does not resolve: how does the recipient learn of the note with which they were paid? The sender must deliver the note to them, privately, over the same public chain. Orchard achieves this with *in-band secret distribution*: every Action carries, beside its commitment, a ciphertext only the recipient can open.

Encryption (sender side). Alice holds Bob’s address (d, pk_d) , with $\text{pk}_d = [\text{ivk}] \text{g}_d$ and $\text{g}_d = \text{GroupHash}^{\text{Pallas}}(\text{Orchard-gd}, d)$ (§2.4). She forms the *note plaintext* $\text{np} = (0x02, d, v, \text{rseed}, \text{memo})$, where the 32-byte rseed is the master randomness from which she derives the note’s rcm , ψ , and the ephemeral secret below. She then runs a one-shot Diffie–Hellman to a *fresh* key:

$$\text{esk} \text{ (derived from } \text{rseed}), \quad \text{epk} := [\text{esk}] \text{g}_d, \quad \text{s} := [\text{esk}] \text{pk}_d, \quad K_{\text{enc}} := \text{KDF}(\text{s}, \text{epk}),$$

and seals the plaintext under an AEAD (ChaCha20-Poly1305), $C_{\text{enc}} := \text{Enc}_{K_{\text{enc}}}(\text{np})$. Because $\text{pk}_d = [\text{ivk}] \text{g}_d$, Bob can reach the same secret from his side as $[\text{ivk}] \text{epk} = [\text{ivk}][\text{esk}] \text{g}_d = [\text{esk}] \text{pk}_d$, so exactly the holder of ivk can recover K_{enc} . A second ciphertext C_{out} , sealed under a key derived from the sender’s ovk , carries $(\text{pk}_d, \text{esk})$ so that the *sender* — or an auditor holding ovk — can re-derive s and read the note as well. The Action publishes $(\text{epk}, C_{\text{enc}}, C_{\text{out}})$ alongside $\text{cv}^{\text{net}}, \text{nf}, \text{rk}, \text{cmx}$.

Trial decryption (recipient side). Nothing on chain marks which Action pays Bob, so his wallet *trial-decrypts every* Orchard output: for each it recomputes $\text{s} := [\text{ivk}] \text{epk}$, $K_{\text{enc}} := \text{KDF}(\text{s}, \text{epk})$, and $\text{np} := \text{Dec}_{K_{\text{enc}}}(C_{\text{enc}})$. The AEAD tag makes an incorrect guess fail cleanly (it returns $\perp$), so a successful open *is* the detection that the note was his. He then re-derives the full note from (d, v, rseed) — taking $\rho := \text{nf}_{\text{old}}$ from the same Action — and performs the two checks that close the obvious forgeries: (i) that he formed the ephemeral key honestly, $[\text{esk}] \text{g}_d = \text{epk}$ for the esk re-derived from rseed (the ZIP-212 check, defeating a mauled epk); and (ii) that the note is indeed

the one the Action committed on chain, $\text{Extract}(\text{NoteCommit}_{\text{rcm}}(\dots)) = \text{cmx}$. Only when both hold does he accept. A sender can therefore never make Bob accept a note inconsistent with the commitment.

Wallet synchronisation, including from cold storage. This is the operation a wallet performs when it restores from a seed. From the account’s “birthday” height it walks the chain forward and trial-decrypts every Action; a light wallet does so against *compact* blocks, which carry only the 52 bytes of C_{enc} that cover (d, v, rseed) , together with epk and cmx . The wallet records each note that opens *with its leaf position* — the index at which it appended its cmx , read directly off the block — and that position is precisely what the commitment tree of §2.6 consumes: the wallet *marks* the position in its shard tree and can then build the note’s authentication path on demand, by the root-only assembly described there, without replaying every commitment. To determine which recovered notes remain spendable it additionally requires nk : it computes each note’s nullifier (§2.7) and discards those whose nullifier already appears in the chain’s nullifier set. The end state is a set of unspent notes, each with a value, a position, an authentication path, and a nullifier — exactly what the Action prover of §2.11 consumes.

2.9 Conservation of value: value commitments and the binding signature

The fourth requirement: to guarantee that a transaction creates no value, while concealing every amount. The instrument is the additive homomorphism of Pedersen commitments (the *Crypto Guide*): commit to each amount, and verify that the commitments *sum* to the publicly declared net flow — the individual values remain hidden, while their balance is publicly verifiable.

Definition 2.8 (Net value commitment). Each Action commits to its own value change with a Pedersen commitment

$$\text{cv}^{\text{net}} := [v_{\text{old}} - v_{\text{new}}] V^{\text{Orchard}} + [\text{rcv}] R^{\text{Orchard}} \in \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}}),$$

where v_{old} is the spent value, v_{new} the created value, $\text{rcv} \in \mathbb{F}_{p_{\text{Vesta}}}$ the commitment randomness (a Pallas scalar), and $V^{\text{Orchard}}, R^{\text{Orchard}} \in \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$ are independent Pallas generators.

We now apply the homomorphism. Summing over the m Actions in a transaction,

$$\sum_{i=1}^m \text{cv}^{\text{net},i} = \left[\sum_i (v_{\text{old},i} - v_{\text{new},i}) \right] V^{\text{Orchard}} + [\text{bsk}] R^{\text{Orchard}}, \quad \text{bsk} := \sum_i \text{rcv}_i \in \mathbb{F}_{p_{\text{Vesta}}}.$$

If the transaction is balanced, the bracketed value-component equals the publicly declared net flow $v_{\text{balance}}^{\text{Orchard}}$ (the amount entering or leaving the shielded pool, in the transaction header). The network subtracts the declared part,

$$\text{bvk} := \sum_i \text{cv}^{\text{net},i} - [v_{\text{balance}}^{\text{Orchard}}] V^{\text{Orchard}} \in \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}}),$$

and *if and only if* the transaction balances, the V^{Orchard} -component cancels and $\text{bvk} = [\text{bsk}] R^{\text{Orchard}}$ is a public key whose secret bsk the transaction author knows. Proving knowledge of that secret — via a *binding signature* (RedPallas) under key bvk — is precisely proving that the accounts balance, with no amount revealed. A single signature over the whole transaction additionally binds all its Actions together, so that no one can remove or transplant any of them.

2.10 Authorisation of the spend: RedPallas

One capability remains: the spender must prove that they are *authorised* to spend the note — that they hold ask — and must do so without linking this spend to any other spend by the same key. RedPallas, a re-randomisable Schnorr signature, provides exactly this.

It is the Schnorr template of the *Crypto Guide* on Pallas, with hash H^* (BLAKE2b-512 reduced mod p_{Vesta}). The novel ingredient is *key re-randomisation*: given a fresh randomiser $\alpha \in \mathbb{F}_{p_{\text{Vesta}}}$,

$$\text{rsk} := \text{ask} + \alpha \in \mathbb{F}_{p_{\text{Vesta}}}, \quad \text{rk} := \text{ak} + [\alpha] G^{\text{SpendAuth}} = [\text{rsk}] G^{\text{SpendAuth}} \in \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}}).$$

The spender publishes the *randomised* validating key rk , not the real ak , and signs with rsk . Because α is fresh per spend, rk is uniformly random and unlinkable across spends, yet remains a valid key for the underlying authority. The circuit enforces (check A6) that rk is a re-randomisation of the ak bound into the note’s address, so that a valid Action proves the spender knows ask — without ever revealing ak .

2.11 A single shielded payment, end to end

The components now assemble into a transaction. Consider Alice paying Bob 5 ZEC from a note worth 5 ZEC. The unit of shielded transfer is an *Action*, which simultaneously spends one old note and creates one new note (a deliberately fixed shape that conceals whether a given Action is “really” a spend, an output, or both). This payment is one Action.

1. Bob hands Alice an address. Bob derives a diversified address (d, pk_d) from his ivk (§2.4) and gives it to Alice. It reveals nothing and is unlinkable to his other addresses.

2. Alice builds the Action. Alice owns an old note $\mathbf{n}_{\text{old}}$ worth 5 ZEC, whose commitment cmx_{old} is a leaf of the tree. She:

- computes its nullifier nf_{old} (§2.7) — she will publish this to retire the note;
- constructs Bob’s new note $\mathbf{n}_{\text{new}} := (\text{addr}_{\text{Bob}}, 5, \rho_{\text{new}}, \psi_{\text{new}}, \text{rcm}_{\text{new}})$ with $\rho_{\text{new}} := \text{nf}_{\text{old}}$, and its commitment cmx_{new} (§2.5);
- forms the value commitment cv^{net} with $v_{\text{old}} - v_{\text{new}} = 5 - 5 = 0$ (§2.9);
- selects a randomiser α and forms rk (§2.10);
- reads off a recent anchor and her note’s Merkle path (§2.6).

3. Alice proves the Action statement. She runs the Halo 2 prover on the relation of §2.13, with public inputs $(\text{anchor}, \text{cv}^{\text{net}}, \text{nf}_{\text{old}}, \text{rk}, \text{cmx}_{\text{new}})$ and her note, keys, path, and randomness as private witness. The proof attests, in one shot and revealing nothing: the old note is in the tree (A3), its nullifier follows correctly (A5), she is authorised to spend it (A6, A7), the new note is well-formed (A1, A2), and the value commitment is consistent (A4).

4. Alice assembles and signs the transaction. She encrypts $\mathbf{n}_{\text{new}}$ to Bob (so that only Bob, via his ivk , can detect and open it), attaches a SpendAuthSig under rk , and signs the whole bundle with the binding signature under bvk (§2.9), which proves balance and welds the Action into this transaction.

5. The network verifies. Consensus checks: it recognises the anchor; nf_{old} is not already in the nullifier set (no double-spend); the Halo 2 proof verifies; the SpendAuthSig verifies under rk ; the binding signature verifies under the recomputed bvk (balance). If all pass, it adds nf_{old} to the nullifier set and appends cmx_{new} as a new tree leaf.

6. Bob receives. Scanning with his ivk , Bob trial-decrypts the ciphertext, recovers $\mathbf{n}_{\text{new}}$, and now owns a 5-ZEC note he can later spend by repeating the entire procedure. The chain has gained one nullifier and one commitment; an observer learns that *an* Orchard payment occurred and nothing else — not sender, recipient, or amount.

That sequence constitutes the entire protocol. The formal Action statement below is precisely the list of the in-circuit checks step 3 invokes.

2.12 What a node verifies, without ever observing the note

Note that none of the previous subsection involved the *network*. A validating node holds no viewing key, cannot trial-decrypt, and so never learns the sender, recipient, or amount of any Action. What it *can* do is confirm that the hidden data is internally consistent — and for that it relies entirely on the proof, not on the note. Using only the public fields ($\text{anchor}, \text{cv}^{\text{net}}, \text{nf}_{\text{old}}, \text{rk}, \text{cmx}$) of each Action, a node checks:

- the *Halo 2 proof* of the Action statement (§2.13) — this forces the spent note to be a genuine tree leaf, the nullifier and new commitment to follow correctly, and the keys to align;
- the *anchor* is a Merkle root the node has seen on its accepted main chain (§2.6);
- nf_{old} is *not already* in the nullifier set — the one check that cannot sit inside a single proof, because it concerns the global history (this prevents a double-spend);
- the *spend-authorisation signature* verifies under rk and the *binding signature* under the recomputed bvk (§§2.9–2.10), settling authority and balance.

If every Action passes, the node adds each nf_{old} to the nullifier set and appends each cmx_{new} as a new tree leaf — the two state changes that permit the next spender to prove membership and that make a second spend of the same note fail. Thus the SNARK together with the two signatures enforces a payment’s validity, never the inspection of the note: the node confirms that the accounts balance and that no one forged anything *while learning nothing about the note itself*. This is the precise sense in which the knowledge soundness of the Action SNARK (§3) performs all the work — the network trusts the proof exactly because a real witness extractably backs a valid one.

2.13 The Action statement, formally

Definition 2.9 (Orchard Action statement, NU5/NU6). The Action SNARK proves, with public inputs

$$(\text{anchor}, \text{cv}^{\text{net}}, \text{nf}_{\text{old}}, \text{rk}, \text{cmx}, \text{enableSpend}, \text{enableOutputs})$$

and the note, keys, Merkle path, randomiser α , and randomness as private witness, the conjunction:

[A1] Old-note commitment integrity — the spent note is well-formed:

$$\text{cm}_{\text{old}} = \text{NoteCommit}_{\text{rcm}_{\text{old}}}(\text{g}_{\text{d}_{\text{old}}}^* \parallel \text{pk}_{\text{d}_{\text{old}}}^* \parallel \text{LE}_{64}(v_{\text{old}}) \parallel \text{LE}_{255}(\rho_{\text{old}}) \parallel \text{LE}_{255}(\psi_{\text{old}})).$$

[A2] New-note commitment integrity, with $\rho_{\text{new}} := \text{nf}_{\text{old}}$: $\text{cm}_{\text{new}} = \text{NoteCommit}_{\text{rcm}_{\text{new}}}(\dots)$ and $\text{cmx} = \text{Extract}(\text{cm}_{\text{new}})$ — the created note is well-formed and its anchor chains to the spent note’s nullifier (Faerie resistance).

[A3] Membership, gated by v_{old} — the spent note is a real leaf: recomputing the Merkle root from $\text{Extract}(\text{cm}_{\text{old}})$ up the witness path of length 32 gives root , and $v_{\text{old}} \cdot (\text{root} - \text{anchor}) = 0$. (The gate by v_{old} permits a zero-value “dummy” spend to skip the membership check, which is how an Action pads with no real input.)

[A4] Net-value commitment integrity — the amounts are consistent and in range: with witnessed (magnitude, sign), $v_{\text{old}} - v_{\text{new}} = \text{magnitude} \cdot \text{sign}$, $\text{cv}^{\text{net}} = [v_{\text{old}} - v_{\text{new}}]V^{\text{Orchard}} + [\text{rcv}]R^{\text{Orchard}}$, $v_{\text{old}}, v_{\text{new}} \in [0, 2^{64})$, $\text{sign} \in \{-1, +1\}$.

[A5] Nullifier integrity — the published nullifier is indeed this note’s: $\text{nf}_{\text{old}} = \text{Extract}([F_{\text{nk}}(\rho_{\text{old}}) + \psi_{\text{old}}]G^{\text{nf}} + \text{cm}_{\text{old}})$.

[A6] Spend authority — the published key is a re-randomisation of the note owner’s: $\text{rk} = \text{ak} + [\alpha]G^{\text{SpendAuth}}$.

[A7] Address integrity — the keys, address, and nullifier key all belong to one identity: $\text{ivk} = \text{ShortCommit}_{\text{rivk}}(\text{ak}^*, \text{nk})$ and $\text{pk}_{\text{dold}} = [\text{ivk}]g_{\text{dold}}$.

[A8] / [A9] Spend / output gating: $v_{\text{old}} \cdot (1 - \text{enableSpend}) = 0$ and $v_{\text{new}} \cdot (1 - \text{enableOutputs}) = 0$ — permitting consensus to disable spends or outputs bundle-wide.

Each conjunct is one entry from the four-requirement map of §2.1: A1–A2 represent notes, A3 proves membership, A5 (with the chain’s nullifier set) prevents double-spending, A4 conserves value, and A6–A7 authorise the spender. This is the relation $\mathcal{R}_{\text{Orchard}}$; Halo 2 produces one proof of it per Action. §2.14 sketches how the Action circuit arithmetises this relation, and §3 shows how soundness of the relation yields the protocol’s security properties.

2.14 The Action circuit: arithmetisation of the relation

The Action statement of §2.13 is a *relation*; what Alice actually executes is a *circuit* that proves membership in it in zero knowledge. Halo 2 (the *Halo 2 Guide*) proves satisfiability of a fixed PLONKish circuit, so the task of the application layer is to encode the conjunction A1–A9 as one such circuit — the *Action circuit*, identical for every Action — supplied the note, keys, Merkle path, randomiser, and randomness as private witness. We sketch this encoding here; the gate-level details appear in the *halo2 Book* and the *Orchard Book*.

We assemble the circuit from a small set of reusable *gadgets* (“chips”), each a pre-wired block of custom gates and lookups realising one operation. The Orchard designers *chose* the primitives so that these gadgets are inexpensive under PLONKish arithmetisation; this motivates the use of Sinsemilla and Poseidon rather than a bit-oriented hash.

The ECC gadget. Implements the Pallas group law with the *complete* addition formulas (no edge-case failures, §2.2), together with variable- and fixed-base scalar multiplication (the former exploiting the j -invariant-0 endomorphism, the latter using precomputed fixed-base tables) and on-curve witnessing. It realises every group operation in the statement: $\text{ak} = [\text{ask}]G^{\text{SpendAuth}}$, the re-randomisation $\text{rk} = \text{ak} + [\alpha]G^{\text{SpendAuth}}$ (A6), the transmission key $\text{pk}_{\text{d}} = [\text{ivk}]g_{\text{d}}$ (A7), the net value commitment $\text{cv}^{\text{net}} = [v_{\text{old}} - v_{\text{new}}]V^{\text{Orchard}} + [\text{rcv}]R^{\text{Orchard}}$ (A4), and the point $[F_{\text{nk}}(\rho) + \psi]G^{\text{nf}} + \text{cm}$ inside the nullifier (A5).

The Sinsemilla gadget. Implements Sinsemilla (§2.3) by realising its generator table $P[m]$ as a Halo 2 *lookup* (the lookup argument of the *Halo 2 Guide*): each 10-bit message chunk costs one table lookup and two incomplete additions. On it rest the two composed commitment gadgets the Orchard Book names explicitly — **NoteCommit**, the in-circuit NoteCommit proving A1 and A2, and **Commitlvk**, the in-circuit short commitment proving $\text{ivk} = \text{ShortCommit}_{\text{rivk}}(\text{ak}^*, \text{nk})$ in A7 — each bundling the Sinsemilla rounds with the bit-decomposition and canonicity checks of their field inputs.

The Merkle-path gadget. Stacks 32 layer-tagged MerkleCRH hashes (themselves Sinsemilla short hashes, §2.6); at each layer a conditional swap orders the running node and its witnessed sibling by one bit of the leaf position, and the gadget asserts the recomputed root equal to the public anchor — check A3, gated by v_{old} so that a dummy spend may skip it.

The Poseidon gadget. Implements the Poseidon permutation (*Crypto Guide*), whose only non-linearity is $x \mapsto x^5$, to evaluate the nullifier PRF $F_{\text{nk}}(\rho) = \text{PoseidonHash}(\text{nk}, \rho)$ cheaply in-circuit; its output feeds the ECC gadget for A5.

Decomposition, range, and boolean gadgets. Lookup-based gadgets split field elements into the bit-chunks the other gadgets consume and range-check values: that $v_{\text{old}}, v_{\text{new}} \in [0, 2^{64}]$ and $\text{sign} \in \{-1, +1\}$ (A4), that point and field encodings are canonical, and that the selectors $\text{enableSpends}, \text{enableOutputs}$ are boolean (A8, A9).

Structure of the proof. These gadgets compose into a single fixed circuit whose native field is the Pallas base field $\mathbb{F}_{p_{\text{Pallas}}}$ — the field in which Pallas coordinates reside, so that the ECC gadget runs natively — proved with the Halo 2 inner-product argument over the Pasta cycle. Every Action, whatever it spends or creates, is one instance of this same circuit, yielding a constant-size proof that a node checks (§2.12) from the public inputs alone; the accumulation machinery of the *Halo 2 Guide* then lets a node verify many such proofs at amortised cost approaching that of a single verification.

3 End-to-end security: derivation of Orchard’s properties from the SNARK

The Halo 2 SNARK on the Action relation, together with the algebraic structure of the Orchard primitives, produces the security properties that render shielded payments functional. We establish those connections below.

3.1 Guarantees provided by a valid Action proof

A SNARK π accepted on public inputs ($\text{anchor}, \text{cv}^{\text{net}}, \text{nf}_{\text{old}}, \text{rk}, \text{cmx}, \dots$) testifies, with overwhelming probability, that the prover knows a witness satisfying A1–A9. By knowledge soundness (the *Halo 2 Guide*, the *Halo 2 Guide*) of Halo 2, an extractor can recover this witness from any successful prover. The Orchard security properties (Theorem-Application chain below) each follow from one or more conjuncts of A1–A9 plus the algebraic properties of the primitives.

3.2 Reduction of Sinsemilla collision-resistance to DL on Pallas

Proposition 3.1 (Sinsemilla CR $\Leftarrow$ DL). *A collision $M \neq M'$ with $\text{Sinsemilla}_D(M) = \text{Sinsemilla}_D(M')$ yields a non-trivial DL relation among the generators $\{P[0], \dots, P[2^k - 1]\}$.*

Sketch. Unfolding the Sinsemilla update (in the case no incomplete-addition exception occurs), with ℓ the number of k -bit chunks:

$$\text{Sinsemilla}_D(M) = 2^\ell \cdot Q + \sum_{i=1}^{\ell} 2^{\ell-i} \cdot P[m_i].$$

A collision gives $\sum_i 2^{\ell-i} (P[m_i] - P[m'_i]) = 0$, i.e. a non-trivial linear relation over $\mathbb{F}_q$ on the generators. Since hash-to-curve modelled as a random oracle samples these, by DL-Rel (the *Crypto Guide*) no efficient adversary produces such a relation without solving DL. The incomplete-addition exceptions are themselves non-trivial DL relations among the same generators and so also negligible. $\square$

Sinsemilla CR is the primitive ingredient for note-commitment binding: two distinct notes cannot share a commitment, so A1 and A2 are genuine integrity statements about cm_{old} and cm_{new} . Combined with knowledge soundness, this entails that the prover knows the specific opening of each commitment.

3.3 Nullifier uniqueness

Proposition 3.2 (Nullifier uniqueness). *Two distinct notes $\mathbf{n} \neq \mathbf{n}'$ cannot share a nullifier except with probability negligible in the DL hardness of Pallas (treating Poseidon outputs as random scalars under the PRF assumption).*

Sketch. A collision $\text{nf}(\mathbf{n}) = \text{nf}(\mathbf{n}')$ in x -coordinate gives $[F_{\text{nk}}(\rho) + \psi]G + \text{cm} = \pm([F_{\text{nk}}(\rho') + \psi']G + \text{cm}')$, a non-trivial linear relation among G, cm, cm' . Since cm, cm' are themselves Pedersen-style commitments to distinct notes with hidden trapdoors, finding such a relation reduces to DL on Pallas. $\square$

A5 (nullifier integrity) plus knowledge soundness ensures that nf_{old} in the public inputs is genuinely the nullifier of some specific committed note, not an arbitrary value. The chain’s duplicate-nullifier rejection then prevents double-spending.

3.4 RedPallas unforgeability and re-randomisation

Proposition 3.3 (RedPallas EUF-CMA $\Leftarrow$ DL). *RedPallas is EUF-CMA-secure assuming DL on Pallas in the random-oracle model. The re-randomisation extension preserves security: a forger against rk for adversary-chosen α implies a forger against plain Schnorr on rk.*

Sketch. The standard Pointcheval–Stern forking argument (the *Crypto Guide*, the *Crypto Guide*) reduces a Schnorr forger to a DL adversary. For re-randomisation, the security simulator selects rsk uniformly, sets $\text{rk} := [\text{rsk}]G^{\text{SpendAuth}}$, then produces a consistent (ak, α) by sampling α at random and computing $\text{ak} := \text{rk} - [\alpha]G^{\text{SpendAuth}}$. The adversary’s view of ak is uniform regardless of rsk , so the forging probability against rk matches that against plain Schnorr on a uniform key. $\square$

A6 (spend authority) plus knowledge soundness ensures the prover knows α relating rk to ak ; combined with the SpendAuthSig under rk at the transaction layer, this enforces that the spender knows ask corresponding to ak , hence to the address pk_{dold} via A7.

3.5 Balance preservation via the Pedersen homomorphism

The combination of A4 (per-Action value commitment integrity), the Pedersen homomorphism on cv^{net} , and the binding signature on bvk gives end-to-end balance:

Proposition 3.4 (Balance). *Any accepted Orchard bundle has $\sum_i (v_{\text{old},i} - v_{\text{new},i}) = v_{\text{balance}}^{\text{Orchard}}$ except with probability negligible in the DL hardness of Pallas.*

Sketch. By A4 and knowledge soundness, each $\text{cv}^{\text{net},i}$ commits to the true per-Action delta $v_{\text{old},i} - v_{\text{new},i}$. The bundle sum $\sum_i \text{cv}^{\text{net},i}$ equals $[\sum_i (v_{\text{old},i} - v_{\text{new},i})]V^{\text{Orchard}} + [\sum_i \text{rcv}_i]R^{\text{Orchard}}$ by homomorphism. Consensus computes bvk and checks the binding signature; by unforgeability (Proposition 3.3), a valid binding signature implies the verifier-side bvk equals $[\text{bsk}]R^{\text{Orchard}}$ for some known bsk — which forces the value-component to equal $[v_{\text{balance}}]V^{\text{Orchard}}$, hence the total value flow matches the declared transparent balance. $\square$

This is the fundamental conservation property: shielded transactions cannot mint zatoshi out of nothing.

3.6 Zero knowledge of the SNARK

Halo 2 is statistical zero knowledge in the random-oracle model (the *Crypto Guide*). The simulator on input $x := (\text{anchor}, \text{cv}^{\text{net}}, \text{nf}_{\text{old}}, \text{rk}, \text{cmx}, \dots)$ operates as follows:

1. Sample the Fiat–Shamir challenges $\theta, \beta, \gamma, y, x, x_1, x_3$ and the IPA challenges $u_1, \dots, u_k$ uniformly, programming the random oracle to return these on the relevant transcript prefixes.

2. Sample the final IPA scalar a uniformly.
3. Use the freedom in the blinding terms ℓ_j, r_j (added at each IPA round) and the polynomial blinders (added in phase 1) to satisfy the verifier’s equations algebraically without knowing the witness.

The resulting transcript is statistically indistinguishable from a real proof; the only gap from perfect ZK is the random-oracle programming, which a verifier that observes only the transcript cannot detect.

ZK furnishes Orchard its privacy: the public chain state reveals only the public inputs of each Action; nothing about the spent note, recipient, or value beyond the explicit transparent total leaks.

3.7 Composition of end-to-end soundness

The full SNARG soundness composes three ingredients:

- *PIOP soundness.* Schwartz–Zippel error per polynomial-identity check, summed across constraints. For $\mathbb{F} = \mathbb{F}_{p_{\text{Pallas}}}$ with $|\mathbb{F}| \approx 2^{254}$ and constraint degree $\leq d_{\text{max}}$, the error per check is $\leq d_{\text{max}}/|\mathbb{F}| \approx d_{\text{max}} \cdot 2^{-254}$, overwhelmingly small.
- *PCS knowledge-binding.* Reduces to DL on the IPA curve (the *Halo 2 Guide*), with the accumulation analysis (the *Halo 2 Guide*) preserving soundness across recursion.
- *Fiat–Shamir soundness in the ROM.* Loses a polynomial factor in the prover’s RO-query budget (the *Crypto Guide*); for $Q \leq 2^{80}$ adversary queries this remains negligible.

Each step’s soundness error is negligible at Orchard’s parameters ($|\mathbb{F}| \approx 2^{254}$, $\lambda = 128$); the union bound across steps is itself negligible. Combined with the application-level reductions (Propositions 3.1–3.4), the full chain gives:

Theorem 3.5 (Orchard end-to-end security, informal). *Under DL on Pallas in the random-oracle model, no efficient adversary can (i) produce an accepted Action whose public inputs do not correspond to a real spent note, (ii) cause two Actions to share a nullifier without spending the same note twice, (iii) cause a bundle’s value flow to exceed the transparent balance, or (iv) link a published Action to any specific recipient or value beyond what is explicit in the public inputs.*

The four properties are, respectively, soundness of the Action SNARK, nullifier uniqueness, balance preservation, and zero knowledge. The first three reduce (informally) to DL on Pallas in the ROM; the fourth (privacy) is stronger — it holds *statistically* given the perfectly-hiding Pedersen and Sinsemilla blinders and the random-oracle programming, so it does not rest on the DL assumption at all. This asymmetry is typical of the discrete-log SNARK family: soundness is computational (a cheating prover is only *bounded*, not impossible), whereas zero knowledge is information-theoretic.

4 The Zcash proof-system landscape

This section closes with a survey of the proof systems Zcash has used across its three generations of shielded payments. Concrete trade-offs drove each transition: trusted setup risk, recursion compatibility, post-quantum posture, and on-chain efficiency.

4.1 Sprout (2016–present, deprecated)

The original shielded protocol used a *BCTV14*-style SNARK (Ben-Sasson, Chiesa, Tromer, Virza; USENIX Security 2014), a refinement of the PGHR13 / Pinocchio QAP-based SNARK family (later superseded by Groth16). The curve was BN254 (then called Barreto–Naehrig), a pairing-friendly curve originally believed to provide ~ 128 -bit security but reduced to ~ 100 -bit by the 2016 extended-tower number field sieve advances.

Properties:

- Trusted setup via a per-circuit Powers of Tau ceremony — the original Zcash ceremony of 2016, run as a multi-party computation with public verification.
- Constant-size proofs (~ 290 bytes), constant verifier time.
- Subject to several minor cryptographic debt items (cofactor handling, edge cases in pairing acceptance) addressed in later iterations.

Sprout was the first deployed shielded-transaction system and demonstrated that zk-SNARK-based privacy was feasible at scale, but its trusted setup, slow prover, and integer-arithmetic SHA256 used inside the circuit rendered it a transitional design.

4.2 Sapling (2018–present)

Sapling (network upgrade 1) replaced Sprout’s stack with:

- Groth16 (Groth; EUROCRYPT 2016): a SNARK with 192-byte proofs and three-pairing verification.
- BLS12-381: a pairing-friendly curve targeting ~ 120 -bit security with a base field large enough to support efficient arithmetic.
- Jubjub: a Twisted Edwards curve embedded inside BLS12-381’s scalar field, which serves in-circuit elliptic-curve operations (RedJubjub signatures, ECC-based hashes).
- Bowe–Hopwood Pedersen hash for the Merkle tree CRH, a bit-by-bit Pedersen-based construction.

Sapling addressed many of Sprout’s deficiencies: a more efficient prover (the Sapling proving time on commodity hardware is several seconds, reduced from minutes for Sprout), a smaller proof, and improved in-circuit hashing. It retained, however, per-circuit trusted setup (the Sapling Powers of Tau, run as a multi-party ceremony with many participants), and its proving system admitted no efficient recursion (no known elliptic-curve cycle extends BLS12-381 / Jubjub).

4.3 Orchard (2021–present, NU5)

Orchard (NU5) replaced the Sapling stack with the Halo 2 / Pasta stack developed in the *Halo 2 Guide*. The principal choices are:

- Halo 2: PLONKish PIOP, IPA-PCS, accumulation-friendly. No trusted setup.
- Pasta cycle: Pallas as the application curve, with Vesta as the proving curve in the IPA layer. Designed for recursion.
- Sinsemilla hash: a Pedersen-Hash-like algebraic hash optimised for PLONKish circuits with lookups; slower than Poseidon *inside* a circuit but substantially faster *outside*, which matters for chain sync where most clients do not generate proofs.

- Poseidon: serves select Orchard primitives where in-circuit speed dominates (the nullifier PRF). The incoming viewing key is instead derived via CommitIvk, a Sinsemilla-based short commitment, not Poseidon. Its security rests on cryptanalytic confidence in algebraic-attack resistance rather than on a hardness assumption such as DL.

The shift followed a consistent design principle: “transparent SNARK, recursion-ready” constituted the design target, and the designers engineered the Pasta cycle to fit.

4.4 Comparative summary

	Sprout	Sapling	Orchard
SNARK	BCTV14	Groth16	Halo 2
Curve	BN254	BLS12-381 + Jubjub	Pallas + Vesta
Trusted setup	Per-circuit	Per-circuit (multi-party)	None
Recursion-ready	No	No	Yes (via Pasta cycle)
Proof size	~290 B	192 B	~1.3–3 KB
Verification time	Constant (pairing)	Constant (3 pairings)	$O(\log d)$ amortized

Table 1: Three generations of Zcash shielded-payment proof systems.

The proof size grew between Sapling and Orchard; this is the cost of transparency. The benefit, beyond the removal of trusted setup, is the foundation for recursion: future generations (Tachyon, eventual rollup designs) can build on Orchard’s foundation without revisiting the cryptographic stack.